University of Exeter

ECM2433: The C Family

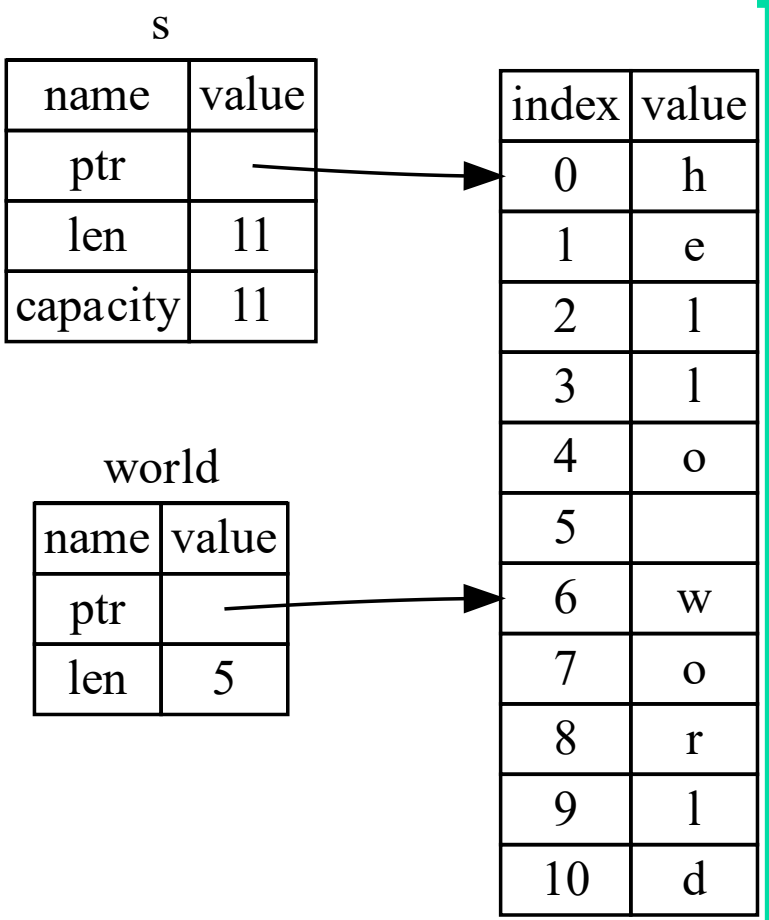
Lecture 04: Slices, Strings, Vec, and Data Modelling

Slices: the concept

```rust
let s = String::from("hello world");

let world = &s[6..];
```

- A slice is a reference to a contiguous sequence of elements
- Written as `&[T]` for a slice of type `T`
- Slices don't own data: they borrow it
- Two components: pointer to first element + length
- [Book ch 4.3]



University of Exeter

Array slices &

```rust
let arr: [i32; 5] = [1, 2, 3, 4, 5];

let slice: &[i32] = &arr[1..4];  // [2, 3, 4]
let slice2 = &arr[..3];          // [1, 2, 3] (from start)
let slice3 = &arr[2..];          // [3, 4, 5] (to end)
let whole = &arr[..];            // [1, 2, 3, 4, 5] (entire array)

println!("Length: {}", slice.len());  // 3
println!("First: {}", slice[0]);      // 2
```

- Range syntax: `start..end` (end exclusive), `start..=end` (end inclusive)
- `&arr[..]` borrows the whole array as a slice
- Slices have `.len()` and support indexing
- [Book ch 4.3]

University of Exeter

String slices &str

```rust
let s: String = String::from("hello world");

let hello: &str = &s[0..5];     // "hello"
let world: &str = &s[6..11];    // "world"

let literal: &str = "hello";    // string literals are &str
```

- String literals are `&str` (stored in the binary)
- Can slice a `String` to get `&str`
- `&str` is pronounced "string slice"
- [Book ch 4.3, 8.2]

University of Exeter

String vs &str

```rust
let owned: String = String::from("hello");  // heap-allocated, growable
let borrowed: &str = "hello";               // pointer + length, immutable view

fn greet(name: &str) {      // accepts both String and &str
    println!("Hello, {name}!");
}

greet(&owned);              // pass &String, coerces to &str
greet(borrowed);            // pass &str directly
```

- `String`: owned, heap-allocated, growable (like C++ `std::string`)
- `&str`: borrowed slice, just pointer + length
- Prefer `&str` for function parameters: it accepts both types
- [Book ch 4.3, 8.2]

University of Exeter

Creating and modifying Strings

```rust
let mut s = String::new();          // empty string
s.push_str("hello");                // append a string slice
s.push(' ');                        // append a single character

let s2 = String::from("world");
let s3 = s + &s2;                   // + consumes s, borrows s2
// s is now invalid, s2 is still valid

let s4 = format!("{} {}!", "hello", "world");  // doesn't consume anything
```

- `String::new()`: create empty, `String::from()`: create from literal
- `push_str(&str)`: append string slice, `push(char)`: append character
- `+` operator: consumes left operand, borrows right
- `format!`: like `println!` but returns a `String`
- [Book ch 8.2]

University of Exeter

String conversions and function parameters

```rust
let s1: String = "hello".to_string();
let s2: String = String::from("hello");

// String to &str
let s: String = String::from("hello");
let slice: &str = &s;            // borrow as slice
let slice2: &str = s.as_str();   // explicit method

// Function that returns owned data
fn make_greeting(name: &str) -> String {
    format!("Hello, {}!", name)
}
```

- `&str` to `String`: `.to_string()` or `String::from()`
- `String` to `&str`: `&` or `.as_str()`
- Return `String` when function creates new data
- Take `&str` when function just reads data
- [Book ch 8.2]

University of Exeter

Vec<T>: creating and modifying

```rust
let mut v: Vec<i32> = Vec::new();    // empty vector
v.push(1);                           // add elements
v.push(2);
v.push(3);

let v2 = vec![1, 2, 3];              // vec! macro: create with initial values
let v3 = vec![0; 5];                 // [0, 0, 0, 0, 0]

let last = v.pop();                  // removes and returns Option<i32>
```

- `Vec<T>`: growable array, like C++ `std::vector<T>`
- `Vec::new()` : create empty, `vec![]` : create with values
- `push`: add to end, `pop`: remove from end (returns `Option`)
- `vec![value; count]` : repeat a value
- [Book ch 8.1]

University of Exeter

Vec<T>: indexing vs get, and iterating

```rust
let mut v = vec![10, 20, 30];

let second = v[1];                  // 20 (panics if out of bounds)
let maybe = v.get(5);               // None (returns Option<&T>)

for x in &v {                       // borrow: v still usable after
    println!("{x}");                // x is &i32
}

for x in &mut v {                   // mutable borrow: can modify elements
    *x += 1;                        // x is &mut i32
}

for x in v {                        // consumes v: v is invalid after
    println!("{x}");                // x is i32 (owned)
}
```

Defining and instantiating structs

```rust
struct Point {
    x: i32,
    y: i32,
}

let p = Point { x: 10, y: 20 };
println!("x: {}, y: {}", p.x, p.y);

let mut q = Point { x: 0, y: 0 };
q.x = 5;                        // can modify if struct is mut
```

- `struct` keyword to define a type with named fields
- Fields have names and types
- Instantiate with `StructName { field: value, ... }`
- Access fields with dot notation
- Entire struct must be `mut` to modify any field -- [Book ch 5.1]

University of Exeter

Field init shorthand and update syntax

```
1 struct Point {
2     x: i32,
3     y: i32,
4 }
5
6 fn make_point(x: i32, y: i32) → Point {
7     Point { x, y }              // shorthand: x means x: x
8 }
9
10 let p1 = Point { x: 10, y: 20 };
11 let p2 = Point { x: 5, ..p1 };  // update syntax: take remaining fields from p1
12                                 // p2 is Point { x: 5, y: 20 }
```

- Field init shorthand: `Point { x, y }` when variable names match field names
- Update syntax: `..other` copies remaining fields from `other` (moves non-`Copy`, copies `Copy`)
- Update syntax must come last in the braces
- [Book ch 5.1]

University of Exeter

Methods with impl

```rust
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn grow(&mut self, amount: u32) {
        self.width += amount;
        self.height += amount;
    }
}

let mut r = Rectangle { width: 10, height: 20 };
println!("Area: {}", r.area());     // 200
r.grow(5);
```

- `impl StructName { }` block contains methods
- `&self`: borrows the instance (read-only)
- `&mut self`: borrows mutably (can modify)
- Call with dot notation: `instance.method()`
- [Book ch 5.3]

Associated functions

```rust
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn new(width: u32, height: u32) -> Self {    // associated function
        Self { width, height }
    }

    fn square(size: u32) -> Self {
        Self { width: size, height: size }
    }
}

let r = Rectangle::new(10, 20);
let s = Rectangle::square(5);
```

- Associated functions: for constructors and other type-level operations
- No `self` parameter: called on the type, not an instance
- Call with `::` syntax: `Rectangle::new(10, 20)`
- `Self` is an alias for the type name inside `impl`
- Convention: `new` for the primary constructor
- [Book ch 5.3]

University of Exeter

Simple enums

```rust
enum Direction {
    North,
    South,
    East,
    West,
}

let d = Direction::North;

fn describe(dir: Direction) {
    // we'll see how to use this with match soon
}
```

- `enum` defines a type with a fixed set of variants
- Access variants with `::` syntax
- Like C enums, but type-safe: can't use integers as directions
- [Book ch 6.1]

University of Exeter

Data-carrying variants

```rust
enum Message {
    Quit,                           // no data
    Move { x: i32, y: i32 },        // named fields (like a struct)
    Write(String),                  // single value
    ChangeColor(i32, i32, i32),     // multiple values
}

let m1 = Message::Quit;
let m2 = Message::Move { x: 10, y: 20 };
let m3 = Message::Write(String::from("hello"));
let m4 = Message::ChangeColor(255, 0, 0);
```

- Variants can hold data of different types and amounts
- Named fields: `Variant { field: Type }`
- Unnamed fields: `Variant(Type)` or `Variant(Type1, Type2)`
- Each variant is like its own struct
- [Book ch 6.1]

University of Exeter

Option<T>

```
1  enum Option<T> {        // defined in standard library
2      Some(T),            // there is a value
3      None,               // there is no value
4  }
5
6  let x: Option<i32> = Some(5);
7  let y: Option<i32> = None;
8
9  let v = vec![10, 20, 30];
10 let first = v.get(0);      // Some(&10)
11 let tenth = v.get(10);     // None
```

- `Option<T>`: represents "maybe a value of type T"
- `Some(value)` : there is a value
- `None`: there is no value
- `Option<T>` is the standard null-free pattern for optional values
- [Book ch 6.1]

University of Exeter

match expressions

```rust
enum Direction { North, South, East, West }

let dir = Direction::North;

let description = match dir {
    Direction::North => {
        println!("Going up!");
        "up"
    }
    Direction::South => "down",
    _ => "sideways",            // matches East and West
};
```

- `match` compares a value against patterns
- Each arm: `pattern => result`
- Multi-line arms use braces `{ }`
- `_` matches anything (like `default` in C)
- Must be exhaustive: all variants must be covered
- [Book ch 6.2]

University of Exeter

Extracting data from variants

```rust
let v = vec![10, 20, 30];
let maybe = v.get(1);            // Option<&i32>

match maybe {
    Some(value) => println!("Found: {}", value),
    None => println!("Not found"),
}

// Also works with custom enums
enum Message {
    Quit,
    Write(String),
}

let msg = Message::Write(String::from("hello"));

match msg {
    Message::Quit => println!("Quit"),
    Message::Write(text) => println!("{}", text),
}
```

- Pattern matching extracts data from variants
- `Some(value)` : binds the inner value to `value`
- `Variant(binding)` : binding is a new variable for that arm
- Most common use: getting values out of `Option`
- [Book ch 6.2]

if let

```rust
let v = vec![10, 20, 30];
let maybe = v.get(1);

// Full match when you only care about one case
match maybe {
    Some(value) => println!("Found: {}", value),
    _ => {}
}

// if let is shorter
if let Some(value) = maybe {
    println!("Found: {}", value)
}

// Can have an else
if let Some(value) = maybe {
    println!("Found: {}", value)
} else {
    println!("Not found")
}
```

- `if let` matches a single pattern
- Shorter than `match` when you only care about one variant
- `else` handles all other cases
- Trade-off: loses exhaustiveness checking
- [Book ch 6.3]

Demo

University of Exeter

Recap

- Slices (`&[T]`, `&str`): views into contiguous data
- `String` vs `&str`: owned vs borrowed; prefer `&str` for parameters
- `Vec<T>`: growable array; `push`, `get`, iterate with `for x in &v`
- Structs: fields, methods (`&self`, `&mut self`), associated functions (`new`)
- Enums: variants with or without data; `Option<T>` for "maybe a value"
- Pattern matching: `match` (exhaustive) and `if let` (single pattern)
- Required reading:
  - Book Chapter 4.3: The Slice Type
  - Book Chapter 5: Using Structs to Structure Related Data
  - Book Chapter 6: Enums and Pattern Matching
  - Book Chapter 8.1-8.2: Vectors and Strings

University of Exeter